字符编码

从按下键盘，到屏幕出现汉字

你遇见过乱码吗？为什么会出现乱码？



同一份文字，为什么会出现乱码？

正常显示

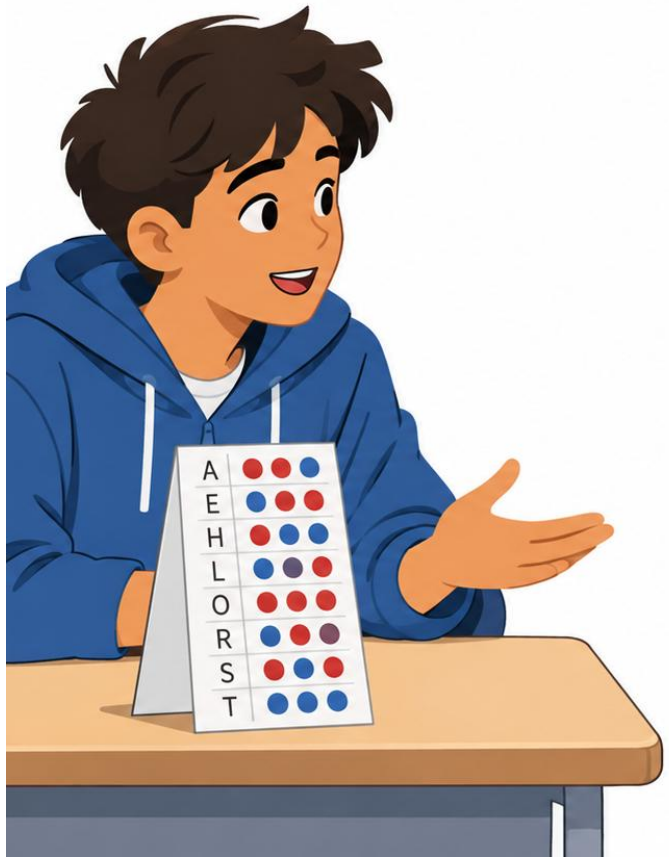
你好

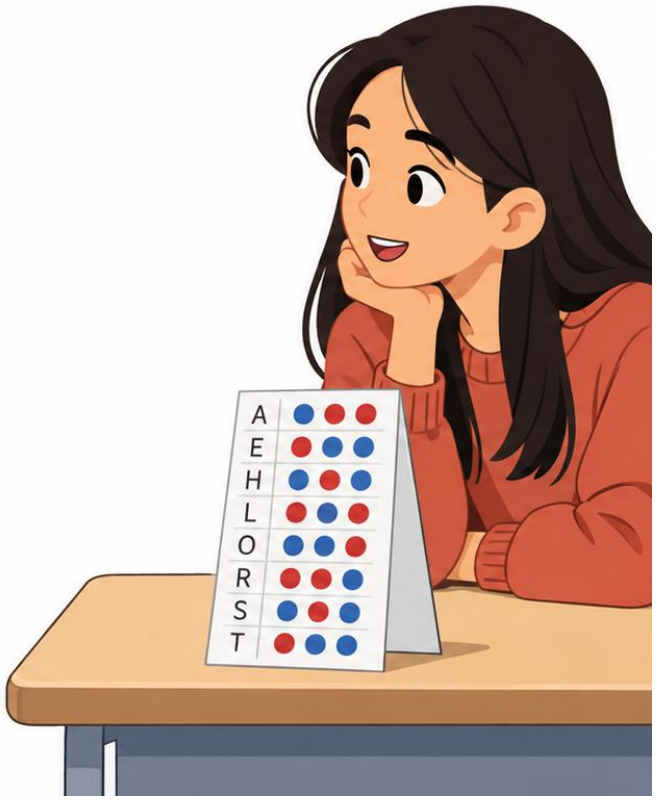


错误显示

浣犲ソ

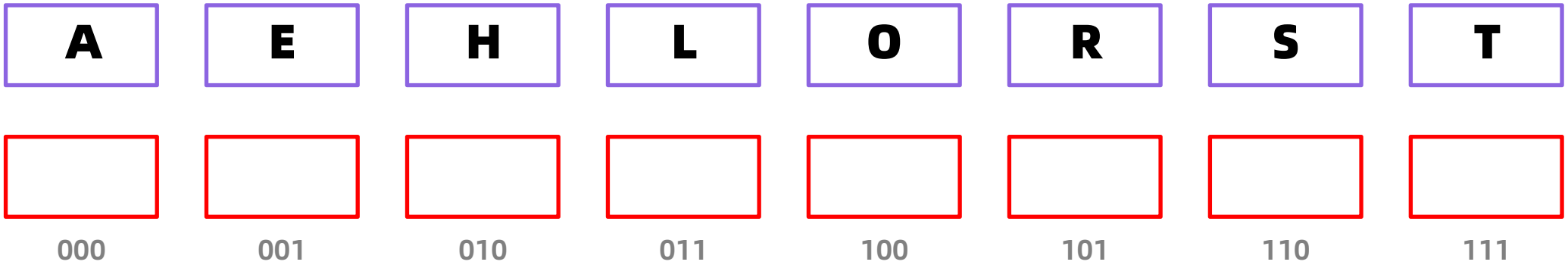
玩个游戏：只用 3 bit，发明一套编码





玩个游戏：只用 3 bit，发明一套编码

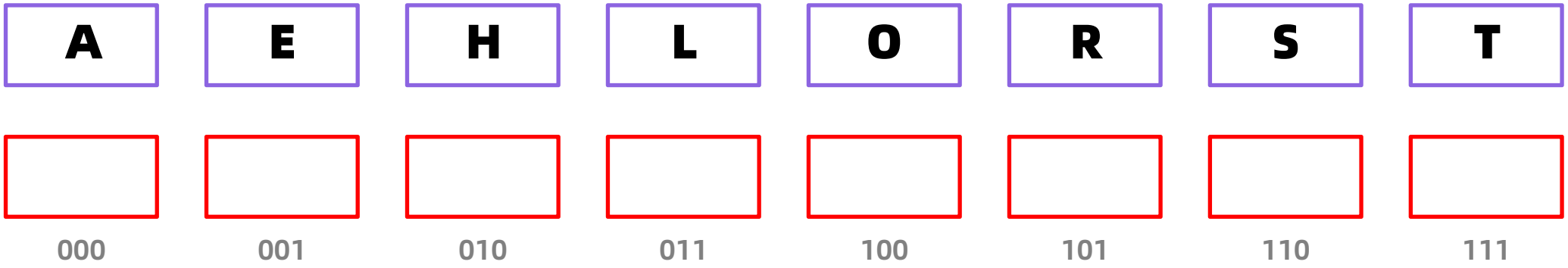
字符集 = {A, E, H, L, O, R, S, T}



环节 1：建立自己的秘密映射表
1）拿出一张白纸
2）把 000 到 111 **打乱**后分给这些字母，并在纸上记录
3）不要给别人看自己设计编码表

玩个游戏：只用 3 bit，发明一套编码

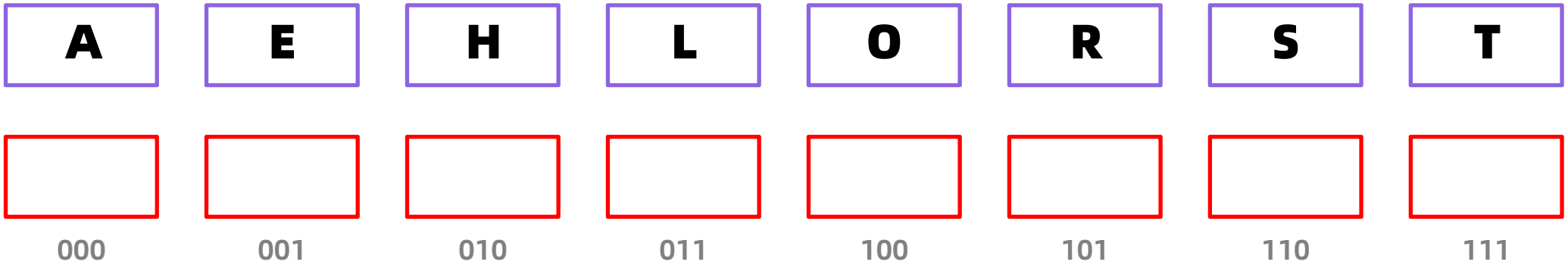
字符集 = { A, E, H, L, O, R, S, T }



环节 2：
1）拿出另一张白纸
2）用自己设计的编码表编码 HELLO，记录bit串在另一张白纸上
3）将bit串给你旁边的同学，看看对方能解读出什么
注：只给 bit串，不给对方你的编码表

玩个游戏：只用 3 bit，发明一套编码

字符集 = { A, E, H, L, O, R, S, T }



环节 3：
1）彼此交换编码表
2）结合编码表，再次解读bit串

思考：从刚才的游戏中，我们学到了什么？

思考：从刚才的游戏中，我们学到了什么？

同一串 bits

010 001 011 011 100



自己的表 → 可能读错

发送者的表 → 恢复 HELLO

bits 本身没有文字意义；共享编码规则后才有确定解释。

思考：从刚才的游戏中，我们学到了什么？

同一串 bits

010 001 011 011 100

→ **自己的表 → 可能读错**

→ **发送者的表 → 恢复 HELLO**

bits 本身没有文字意义；共享编码规则后才有确定解释。

结论：文本交换需要共享字符–编码映射。

如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO

STAR

如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }

如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }

CODE

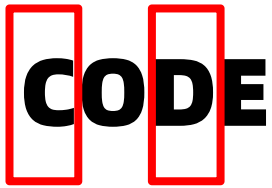
如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }



C、D 不在字符集中

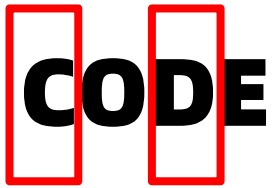
如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }



C、D 不在字符集中

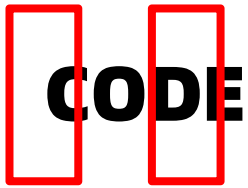
如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }



C、D 不在字符集中

想一想：这对设计一套字符编码有什么启示？

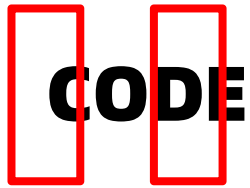
如果要同时编码HELLO和START，但只有 8 个位置，哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → { H, E, L, O }

STAR → { S, T, A, R }

合并 → { A, E, H, L, O, R, S, T }



C、D 不在字符集中

想一想：这对设计一套字符编码有什么启示？

结论：设计字符编码，要先确定字符集，再为字符建立唯一编码映射，并且文本交换时需要共享字符集和字符–编码映射。

现实世界谁来制定这张大家都能查的表？

现实世界谁来制定这张大家都能查的表？

我们先来看看英文世界的编码表。

美国信息技术交换标准码p20
American Standard Code for Information Interchange
ASCII

活动：请你读一读ASCII 表？

B \ b7 ---------->	0	0	0	0	1	1	1	1
I \ b6 ---------->	0	0	1	1	0	0	1	1
T \ b5 -------->	0	1	0	1	0	1	0	1
S / COLUMN->	0	1	2	3	4	5	6	7
b4 b3 b2 b1 / ROW								
0 0 0 0 0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0 2	STX	DC2	"	2	B	R	b	r
0 0 1 1 3	ETX	DC3	#	3	C	S	c	s
0 1 0 0 4	EOT	DC4	$	4	D	T	d	t
0 1 0 1 5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0 6	ACK	SYN	&	6	F	V	f	v
0 1 1 1 7	BEL	ETB	'	7	G	W	g	w
1 0 0 0 8	BS	CAN	(	8	H	X	h	x
1 0 0 1 9	HT	EM	)	9	I	Y	i	y
1 0 1 0 10	LF	SUB	*	:	J	Z	j	z
1 0 1 1 11	VT	ESC	+	;	K	[	k	{
1 1 0 0 12	FF	FS	,	<	L	\	l	|
1 1 0 1 13	CR	GS	-	=	M	]	m	}
1 1 1 0 14	SO	RS	.	>	N	^	n	~
1 1 1 1 15	SI	US	/	?	O	_	o	DEL

1）有哪些字符？
2）字符A 的编码是什么？
3）一共有多少个字符？

source: https://www.rfc-editor.org/info/rfc20/

B \ b7 ---------->	0	0	0	0	1	1	1	1
I \ b6 ---------->	0	0	1	1	0	0	1	1
T \ b5 -------->	0	1	0	1	0	1	0	1
S / COLUMN->	0	1	2	3	4	5	6	7
b4 b3 b2 b1 / ROW								
0 0 0 0 0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0 2	STX	DC2	"	2	B	R	b	r
0 0 1 1 3	ETX	DC3	#	3	C	S	c	s
0 1 0 0 4	EOT	DC4	$	4	D	T	d	t
0 1 0 1 5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0 6	ACK	SYN	&	6	F	V	f	v
0 1 1 1 7	BEL	ETB	'	7	G	W	g	w
1 0 0 0 8	BS	CAN	(	8	H	X	h	x
1 0 0 1 9	HT	EM	)	9	I	Y	i	y
1 0 1 0 10	LF	SUB	*	:	J	Z	j	z
1 0 1 1 11	VT	ESC	+	;	K	[	k	{
1 1 0 0 12	FF	FS	,	<	L	\	l	|
1 1 0 1 13	CR	GS	-	=	M	]	m	}
1 1 1 0 14	SO	RS	.	>	N	^	n	~
1 1 1 1 15	SI	US	/	?	O	_	o	DEL

1）有哪些字符？

source: https://www.rfc-editor.org/info/rfc20/

ASCII里，为什么有些“字符”看不见？



看不见 ≠ 没有意义

BEL 响铃　　CR 回车　　LF 换行

控制字符说明：早期字符编码服务的不只是“显示文字”，也服务于通信与设备控制。

B \ b7 ---------->	0	0	0	0	1	1	1	1
I \ b6 ---------->	0	0	1	1	0	0	1	1
T \ b5 -------->	0	1	0	1	0	1	0	1
S / COLUMN->	0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

2）字符A 的编码是什么？

source: https://www.rfc-editor.org/info/rfc20/

B \ b7 ---------->	0	0	0	0	1	1	1	1
I \ b6 ---------->	0	0	1	1	0	0	1	1
T \ b5 -------->	0	1	0	1	0	1	0	1
S / COLUMN->	0	1	2	3	4	5	6	7
b4 b3 b2 b1 / ROW								
0 0 0 0 / 0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1 / 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0 / 2	STX	DC2	"	2	B	R	b	r
0 0 1 1 / 3	ETX	DC3	#	3	C	S	c	s
0 1 0 0 / 4	EOT	DC4	$	4	D	T	d	t
0 1 0 1 / 5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0 / 6	ACK	SYN	&	6	F	V	f	v
0 1 1 1 / 7	BEL	ETB	'	7	G	W	g	w
1 0 0 0 / 8	BS	CAN	(	8	H	X	h	x
1 0 0 1 / 9	HT	EM	)	9	I	Y	i	y
1 0 1 0 / 10	LF	SUB	*	:	J	Z	j	z
1 0 1 1 / 11	VT	ESC	+	;	K	[	k	{
1 1 0 0 / 12	FF	FS	,	<	L	\	l	|
1 1 0 1 / 13	CR	GS	-	=	M	]	m	}
1 1 1 0 / 14	SO	RS	.	>	N	^	n	~
1 1 1 1 / 15	SI	US	/	?	O	_	o	DEL

2）字符A 的编码是什么？1000001

```
(base) → repo ascii
Usage: ascii [-adxohv] [-t] [char-alias...]
   -t = one-line output  -a = vertical format
   -d = decimal table  -o = octal table  -x = hex table  -b binary table
   -h = this help screen -v = version information
Prints all aliases of an ASCII character. Args may be chars, C \-escapes,
English names, ^-escapes, ASCII mnemonics, or numerics in decimal/octal/hex.

Dec Hex    Dec Hex    Dec Hex  Dec Hex  Dec Hex  Dec Hex   Dec Hex   Dec Hex
  0 00 NUL  16 10 DLE  32 20    48 30 0  64 40 @  80 50 P   96 60 `  112 70 p
  1 01 SOH  17 11 DC1  33 21 !  49 31 1  65 41 A  81 51 Q   97 61 a  113 71 q
  2 02 STX  18 12 DC2  34 22 "  50 32 2  66 42 B  82 52 R   98 62 b  114 72 r
  3 03 ETX  19 13 DC3  35 23 #  51 33 3  67 43 C  83 53 S   99 63 c  115 73 s
  4 04 EOT  20 14 DC4  36 24 $  52 34 4  68 44 D  84 54 T  100 64 d  116 74 t
  5 05 ENQ  21 15 NAK  37 25 %  53 35 5  69 45 E  85 55 U  101 65 e  117 75 u
  6 06 ACK  22 16 SYN  38 26 &  54 36 6  70 46 F  86 56 V  102 66 f  118 76 v
  7 07 BEL  23 17 ETB  39 27 '  55 37 7  71 47 G  87 57 W  103 67 g  119 77 w
  8 08 BS   24 18 CAN  40 28 (  56 38 8  72 48 H  88 58 X  104 68 h  120 78 x
  9 09 HT   25 19 EM   41 29 )  57 39 9  73 49 I  89 59 Y  105 69 i  121 79 y
 10 0A LF   26 1A SUB  42 2A *  58 3A :  74 4A J  90 5A Z  106 6A j  122 7A z
 11 0B VT   27 1B ESC  43 2B +  59 3B ;  75 4B K  91 5B [  107 6B k  123 7B {
 12 0C FF   28 1C FS   44 2C ,  60 3C <  76 4C L  92 5C \  108 6C l  124 7C |
 13 0D CR   29 1D GS   45 2D -  61 3D =  77 4D M  93 5D ]  109 6D m  125 7D }
 14 0E SO   30 1E RS   46 2E .  62 3E >  78 4E N  94 5E ^  110 6E n  126 7E ~
 15 0F SI   31 1F US   47 2F /  63 3F ?  79 4F O  95 5F _  111 6F o  127 7F DEL
```

source: https://www.rfc-editor.org/info/rfc20/

B \ b7 ---------->	0	0	0	0	1	1	1	1
I \ b6 ---------->	0	0	1	1	0	0	1	1
T \ b5 -------->	0	1	0	1	0	1	0	1
S \ COLUMN->	0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

3）一共有多少个字符?

source: https://www.rfc-editor.org/info/rfc20/

B / I / T / S	b7	0	0	0	0	1	1	1	1
	b6	0	0	1	1	0	0	1	1
	b5	0	1	0	1	0	1	0	1
	COLUMN →	0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

3）一共有多少个字符？2^7 = 128

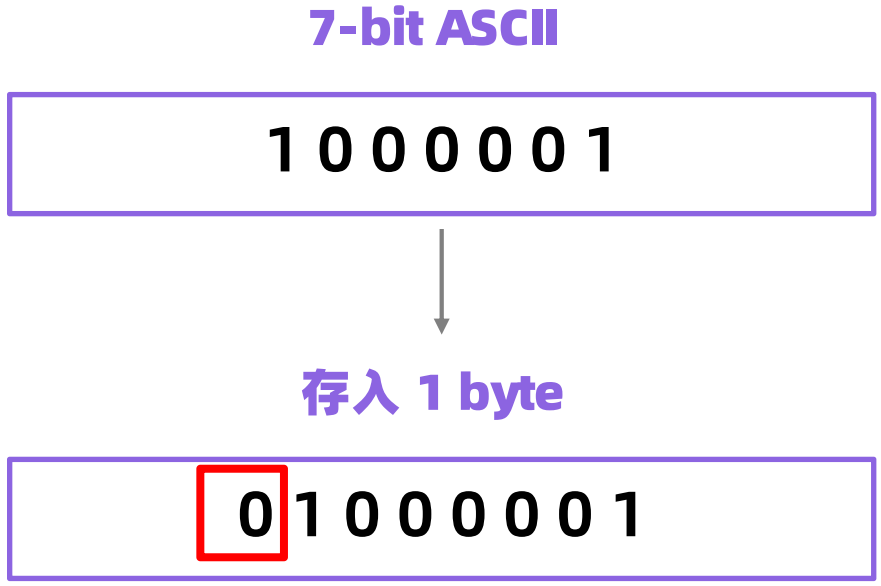
source: https://www.rfc-editor.org/info/rfc20/

ASCII 是 7 bit，为什么常看到 8 bit?



ASCII 是 7 bit，为什么常看到 8 bit?

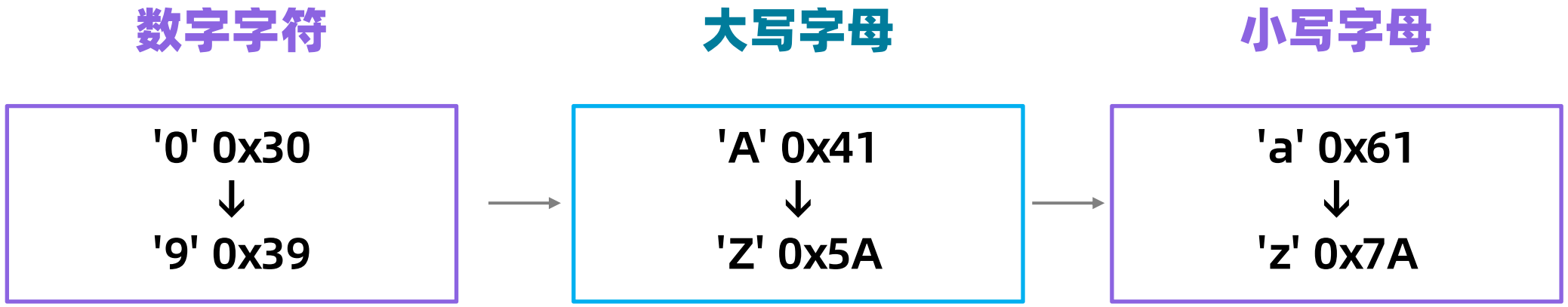
Character:	A
Decimal code:	65
Hex code:	0x41
ASCII 7-bit code :	1000001
Stored in 1 byte :	01000001
MSB:	0



重点：标准的ASCII码为7位二进制编码，存储时占用一个字节，最高位为0。p20

数字、大写、小写在 ASCII 中谁更小？

数字、大写、小写在 ASCII 中谁更小？



重点：ASCII编码值顺序：数字字符 < 大写字母 < 小写字母。

如何设计汉字编码？

如何设计汉字编码？ASCII 的哪些思想还能用？

如何设计汉字编码？ASCII 的哪些思想还能用？

ASCII 给出的设计问题

先确定字符集

每个字符唯一映射

ASCII 有 128 个位置

常用 1 byte 存放

如何设计汉字编码？ASCII 的哪些思想还能用？

ASCII 给出的设计问题		汉字编码必须重新回答
先确定字符集	→	**哪些汉字进入字符集？**
每个字符唯一映射	→	**每个汉字对应什么编码？**
ASCII 有 128 个位置	→	**容量够吗？**
常用 1 byte 存放	→	**1 byte 够吗？**

字符集与映射可以借鉴；容量与存储必须重新设计。

汉字写进文件以后是什么？



汉字写进文件以后是什么？

样本：A中B国C文

41	D6 D0	42	B9 FA	43	CE C4
A	中	B	国	C	文

“中” → D6 D0

ASCII 字符
1 byte

GB2312 样本汉字
2 bytes

D6 D0 为什么能代表“中”？

D6 D0 为什么能代表“中”？

“中”	国标码	56	50
		+80	+80
	机内码	D6	D0

MSB 区分

ASCII
0xxxxxxx

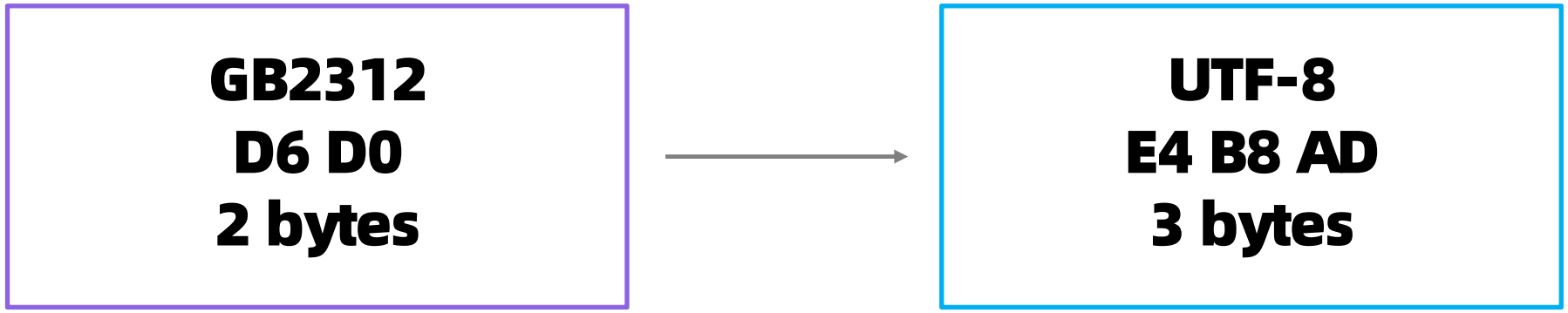
汉字
1xxxxxxx 1xxxxxxx

这里的“机内码”只限于 GB2312 双字节模型。

汉字真的总是占 2 bytes 吗？

汉字真的总是占 2 bytes 吗？

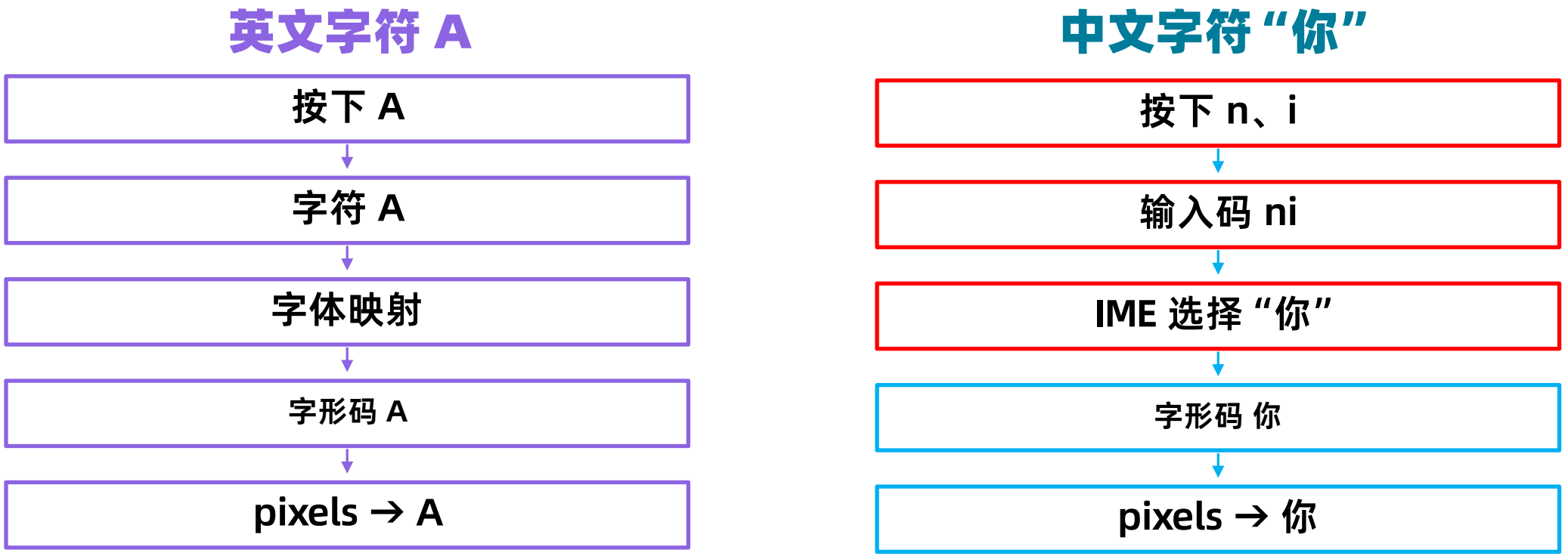
同一个字符：“中”



“汉字占 2 bytes”不是汉字本身的属性。

英文字符和汉字，从键盘到屏幕有什么不同？

英文字符和汉字，从键盘到屏幕有什么不同？



中文输入多了“输入码 → IME → 选字”；
显示阶段仍要经过 字形码 → pixels。

中文编码标准为什么不断扩展？



中文编码标准为什么不断扩展？



如果字符不在字符集中，系统就无法稳定交换它。

如何给全球字符进行编码？第一步要做什么？

A

U+0041

中

U+4E2D

ש

U+05E9

U+1F600

Unicode：给全球字符分配统一码位

A	中	ש	𝄞
U+0041	U+4E2D	U+05E9	U+1F600

Unicode 先回答：“这个字符的全球统一编号是多少？”

注意：码位不是字节。下一步才是 UTF。

有了 Unicode，为什么还会有 UTF-8 / UTF-16 / UTF-32?

Unicode 码位	UTF-8	UTF-16BE	UTF-32BE
U+4E2D	E4 B8 AD	4E 2D	00 00 4E 2D

有了 Unicode，为什么还会有 UTF-8 / UTF-16 / UTF-32?

Unicode 码位	UTF-8	UTF-16BE	UTF-32BE
U+4E2D	E4 B8 AD	4E 2D	00 00 4E 2D

Unicode 决定“编号”，UTF 决定“字节”。

同一个码位，可以有不同的字节写法。

UTF-8、UTF-16、UTF-32 的差异，本质是“用多少字节”

字符	UTF-8	UTF-16	UTF-32
A	1 字节 41	2 字节 00 41	4 字节 00 00 00 41
语	3 字节 E8 AF AD	2 字节 8B ED	4 字节 00 00 8B ED
	4 字节 F0 9F 98 80	4 字节 D8 3D DE 00	4 字节 00 01 F6 00

同一个 Unicode 码位，不同 UTF 形式会有不同空间成本。

UTF-8、UTF-16、UTF-32，谁更优秀？

UTF-8

传输与存储

兼容 ASCII

UTF-16

平台历史

兼顾 BMP 字符

UTF-32

定长简单

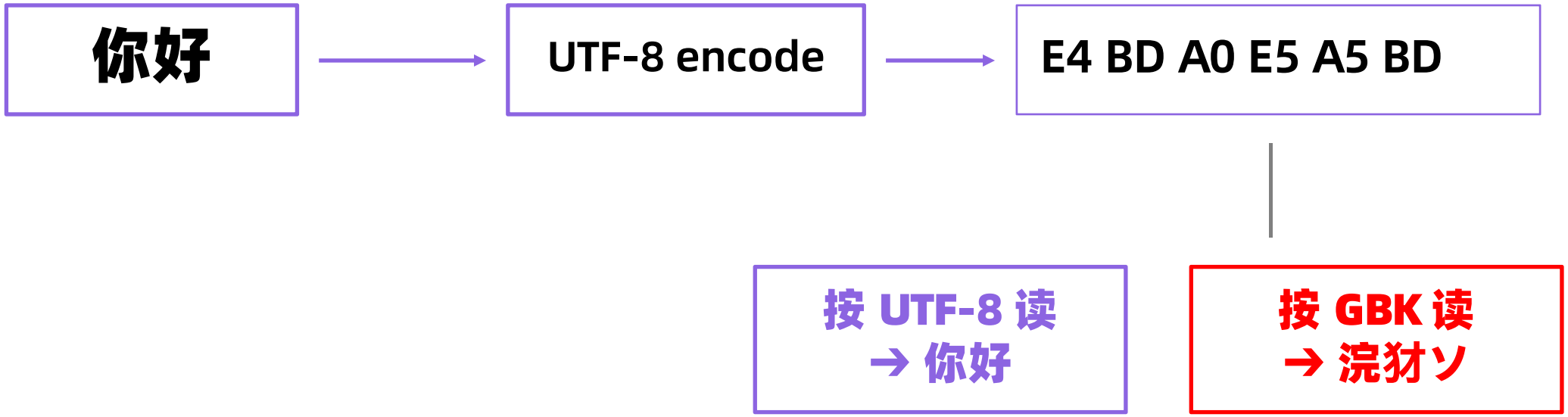
空间成本高

没有绝对优秀，只有场景权衡。

同一批 bytes，为什么会变成乱码？



同一批 bytes，为什么会变成乱码？

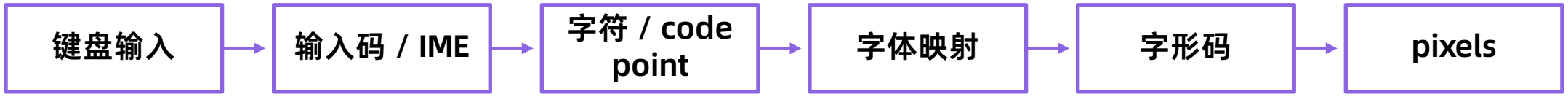


同一批 bytes，使用不匹配的 decode 规则，就可能显示成乱码。

现在能重新解释整个过程吗？

现在能重新解释整个过程吗？

输入并显示



保存并读取



乱码定位：保存并读取路径中，decode 使用了不匹配的规则。

用三句话解释字符编码

1. 一串 bits 为什么不能自己决定文字意义？

2. Unicode 码位与三种 UTF 各负责什么？

3. bytes 没坏时，为什么仍可能出现乱码？